



Universidade do Minho

Braga, Portugal

TRABALHO PRÁTICO - RELATÓRIO

COMPUTAÇÃO GRÁFICA

Fase IV - Normais e Coordenadas de textura

Grupo 37

Engenharia Informática 2025/26

Equipa de Trabalho:

A106936 - Duarte Escairo Brandão Reis Silva

A106932 - Luís António Peixoto Soares

A106856 - Tiago Silva Figueiredo

A104704 - Inês Ferreira Ribeiro

31 Maio 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Atualização da Garrafa de Klein	2
2.2. Atualização do formato do ficheiro .3d	3
2.2.1. Geração do raio e centro da esfera no output	3
2.2.1.1. Plano	3
2.2.1.2. Box	4
2.2.1.3. Esfera	4
2.2.1.4. Cone	4
2.2.1.5. Torus	4
2.2.1.6. Cilindro	4
2.2.1.7. Fita de Möbius	4
2.2.1.8. Superfície de Bezier	4
2.2.1.9. Garrafa de Klein	4
2.2.2. Geração de normais e coord. Textura	5
2.2.2.1. Plano	5
2.2.2.2. Box	5
2.2.2.3. Esfera	5
2.2.2.4. Cone	6
2.2.2.5. Cilindro	6
2.2.2.6. Torus	7
2.2.2.7. Fita de Möbius	7
2.2.2.8. Garrafa de Klein	7
2.2.2.9. Superfícies Bézier	8
3. Engine	9
3.1. Extensão no formato XML	9
3.2. Atualização da Câmara Orbital	10
3.3. View Frustum Culling	11
3.4. Picking	12
3.5. Iluminação	13
3.6. Texturas	14
4. Geração do Sistema Solar Dinâmico com Iluminação e Texturas	15
5. Resultados Obtidos	20
6. Conclusão	23

1. Introdução

No âmbito da unidade curricular de Computação Gráfica, foi proposto o desenvolvimento de um projeto cujo objetivo consistiu na implementação de um mini motor gráfico 3D baseado numa estrutura de *scene graph*, o **engine**, e um gerador de modelos 3D, o **generator**, que fornece modelos de exemplo a serem utilizados pelo motor.

Nesta fase, foram introduzidas texturas e iluminação ao projeto, e devido a isso tivemos de adaptar o **generator** de forma a produzir as normais e coordenadas de textura das primitivas no seu respetivo output e também de atualizar o **engine** de forma ao mesmo ser capaz de suportar estas alterações. Assim como nas fases anteriores, tivemos de criar uma cena de demonstração que usasse os conhecimentos adquiridos na fase. A cena desenvolvida foi uma representação dinâmica do sistema solar com iluminação e texturas.

Como extras, atualizámos a câmara orbital de forma à mesma poder orbitar diversos “targets” diferentes, implementámos um view frustum culling com esferas, utilizámos também picking para expandir ainda mais a câmara orbital e, por fim, atualizámos a primitiva garrafa de klein implementada na fase anterior de forma à mesma ser mais configurável.

2. Generator

2.1. Atualização da Garrafa de Klein

Nesta fase resolvemos atualizar esta primitiva de forma a torna-la mais configurável. Para isso adicionamos 3 novos argumentos à sua parametrização, a figura atualizada pode ser gerada pelo **generator** com o seguinte comando:

```
./generator kleinBottle <slices> <stacks> <scale> <tubeRadius> <heightStretch> <dest file>
```

O parâmetro **scale** é um fator de escala aplicado a todas as coordenadas da figura, o que nos permite controlar o tamanho geral da figura, o parâmetro **tubeRadius** dita o raio do tubo que forma a garrafa, e, por fim, o parâmetro **heightStretch** permite ajustar a altura da garrafa ao longo do eixo vertical sem afetar o raio do tubo e o resto da figura.

Assim como foi dito na fase anterior, a parametrização desta figura utiliza dois ângulos $u \in [0, 2\pi]$ e $v \in [0, 2\pi]$, e pode variar entre duas formas distintas dependendo do valor do ângulo u .

Para $u \in [0, \pi[$, as coordenadas de cada ponto passam a ser calculadas por:

$$\begin{aligned} x &= \text{scale} \cdot 3 \cdot \cos(u) \cdot (1 + \sin(u)) + \left(2 \cdot \text{tubeRadius} \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \cos(u) \cdot \cos(v) \\ y &= \text{scale} \cdot 8 \cdot \text{heightStretch} \cdot \sin(u) + \left(2 \cdot \text{tubeRadius} \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \sin(u) \cdot \cos(v) \\ z &= \text{scale} \cdot \left(2 \cdot \text{tubeRadius} \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \sin(v). \end{aligned}$$

Para $u \in [\pi, 2\pi]$, o **heightStretch** mantém-se aplicado no mesmo termo, o **tubeRadius** continua a ser usado nas componentes do tubo e o **scale** continua a atuar globalmente:

$$\begin{aligned} x &= \text{scale} \cdot 3 \cdot \cos(u) \cdot (1 + \sin(u)) + \left(2 \cdot \text{tubeRadius} \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \cos(v + \pi) \\ y &= \text{scale} \cdot 8 \cdot \text{heightStretch} \cdot \sin(u) \\ z &= \text{scale} \cdot \left(2 \cdot \text{tubeRadius} \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \sin(v). \end{aligned}$$

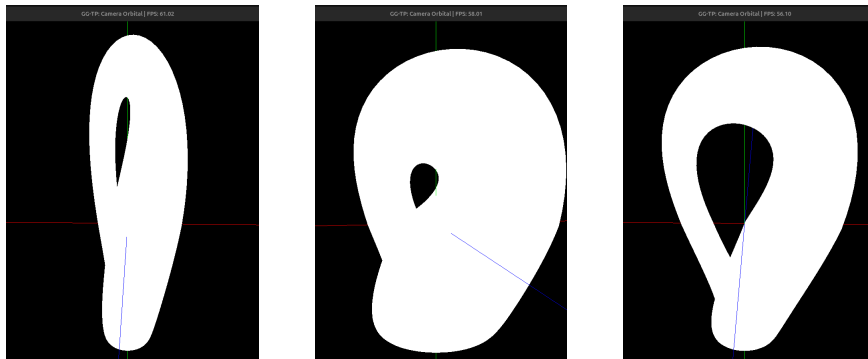


Figura 1: Variações da Garrafa de Klein

2.2. Atualização do formato do ficheiro .3d

Com a introdução da iluminação e das texturas no projeto fomos obrigados a mudar o formato do output do **generator** de forma ao mesmo conter informação referente às normais e às coordenadas de textura. Além disso, de forma a facilitar a implementação do view frustum culling com esferas no **engine**, o output do **generator** também possui informação em relação à esfera usada para determinar se a respetiva primitiva está ou não dentro do frustum e se deve então ser renderizada.

Em baixo, é possível ver um exemplo do novo output dos ficheiros .3d produzidos pelo **generator**:

```
0.707107
0 0 0
6
-0.5 0 -0.5
0.5 0 0.5
0.5 0 -0.5
-0.5 0 -0.5
-0.5 0 0.5
0.5 0 0.5
0 1 0
0 1 0
0 1 0
0 1 0
0 1 0
0 1 0
0 1 0
0 0
1 1
1 0
0 0
0 1
1 1
```

A primeira e segunda linha contém o raio e centro locais da esfera, respetivamente. Na terceira linha está o número de vértices, normais e coordenadas de textura. Por fim, as restantes linhas possuem os vértices, as normais e as coordenadas de textura, respetivamente, onde, neste caso, onde o número da terceira linha é 6, as linhas 4 a 9 possuem os 6 vértices, as linhas 10 a 15 possuem as normais dos 6 vértices e as linhas 16 a 21 possuem as coordenadas de textura.

2.2.1. Geração do raio e centro da esfera no output

Nas secções a baixo é possível ver como foram calculados o raio e centro da esfera usada no view frustum culling para cada primitiva implementada.

2.2.1.1. Plano

O plano é construído na origem, logo o centro é $(0,0,0)$ e o raio é calculado fazendo $\text{length} \cdot \frac{\sqrt{2}}{2}$, onde **length** é um dos parâmetros da primitiva

2.2.1.2. Box

A box é construída na origem, logo o centro é $(0,0,0)$ e o raio é calculado fazendo $\text{length} \cdot \frac{\sqrt{3}}{2}$, onde `length` é um dos parâmetros da primitiva

2.2.1.3. Esfera

A esfera é construída na origem e um dos seus parâmetros é o próprio raio, logo o centro é $(0,0,0)$ e o raio é o parâmetro `radius`.

2.2.1.4. Cone

No caso do cone, o mesmo não é construído centrado na origem, apenas possui a base construída nela, ou seja, é necessário mais cálculos para determinar o centro da esfera. Para realizar este cálculo, foi feita a média de todos os pontos gerados, ou seja, somando (x,y,z) de todos os vértices e dividindo no fim pelo número total de vértices. Já o raio é calculado fazendo $\sqrt{\text{radius} \cdot \text{radius} + \left(\frac{\text{height}}{2}\right) \cdot \left(\frac{\text{height}}{2}\right)}$, onde `radius` e `height` são parâmetros do cone.

2.2.1.5. Torus

O torus também é construído na origem, logo o centro é $(0,0,0)$, e o raio é obtido fazendo a soma `majorRadius` + `minorRadius`, onde ambos são parâmetros desta primitiva.

2.2.1.6. Cilindro

O cilindro também é, mais uma vez, construído na origem, logo o centro é $(0,0,0)$ e, assim como o cone, o raio é calculado com a expressão $\sqrt{\text{radius} \cdot \text{radius} + \left(\frac{\text{height}}{2}\right) \cdot \left(\frac{\text{height}}{2}\right)}$, onde `radius` e `height` são parâmetros do cilindro.

2.2.1.7. Fita de Möbius

Assim como no cone, a fita de Möbius não é construída na origem, mas o centro pode ser calculado fazendo a média de todos os pontos gerados. O raio pode ser calculado usando os parâmetros `radius` e `width` através da expressão `radius` + `width`.

2.2.1.8. Superfície de Bezier

Na superfície de bezier, o centro é calculado da mesma forma que o cone e a fita de Möbius, através da média de todos os pontos gerados. Neste caso, o raio envolve mais cálculos, uma vez que os parâmetros desta primitiva não possuem informação que facilite este cálculo. Este pode ser determinado calculando a distância máxima de um qualquer vértice até ao centro determinado pela média dos pontos gerados.

2.2.1.9. Garrafa de Klein

Na garrafa de klein, o cálculo do centro e do raio é feito da mesma forma que na superfície de bezier.

2.2.2. Geração de normais e coord. Textura

2.2.2.1. Plano

O plano é construído numa grelha regular no plano XZ, com altura constante $y = 0$. Como a superfície é horizontal, a normal é constante para todos os vértices:

$$n = (0, 1, 0)$$

As coordenadas de textura são mapeadas linearmente por célula da grelha:

$$u = \frac{j}{\text{divisions}}$$

$$v = \frac{i}{\text{divisions}}$$

onde, i é o índice da linha e j é o índice da coluna.

2.2.2.2. Box

A caixa é tratada como seis faces independentes, cada uma subdividida em grelha. Em cada face, a normal é constante e perpendicular ao plano da face:

- face superior: $(0, 1, 0)$
- face inferior: $(0, -1, 0)$
- face frontal: $(0, 0, 1)$
- face traseira: $(0, 0, -1)$
- face direita: $(1, 0, 0)$
- face esquerda: $(-1, 0, 0)$

O mapeamento das coordenadas de textura em cada face é local e linear, com parametrização de célula análoga ao plano:

$$u = \frac{j}{\text{divisions}}$$

$$v = \frac{i}{\text{divisions}}$$

Isto garante distribuição uniforme da textura por face, sem dependência das restantes faces.

2.2.2.3. Esfera

As normais da esfera são calculadas diretamente a partir das coordenadas do vértice, uma vez que, para uma esfera centrada na origem, o vetor normal em qualquer ponto da superfície é simplesmente o vetor que liga a origem a esse ponto. Assim, a normal de um vértice (x, y, z) é:

$$n = \left(\frac{x}{\text{radius}}, \frac{y}{\text{radius}}, \frac{z}{\text{radius}} \right)$$

onde radius é o raio da esfera. As coordenadas de textura são calculadas a partir dos ângulos paramétricos α e β , normalizando-os para o intervalo $[0, 1]$:

$$u = \frac{\alpha}{2\pi}$$

$$v = 1 - \frac{\frac{\pi}{2} - \beta}{\pi}$$

A inversão de v garante que a textura é aplicada com a orientação vertical correta.

2.2.2.4. Cone

A base do cone, no plano $y = 0$, recebe normais constantes apontando para baixo:

$$n = (0, -1, 0)$$

As coordenadas de textura da base são calculadas projetando circularmente os pontos sobre o quadrado unitário:

$$u = 0.5 + 0.5 \cos(\alpha)$$

$$v = 0.5 + 0.5 \sin(\alpha)$$

Para a superfície lateral, a normal em cada vértice (x, y, z) é calculada tendo em conta a inclinação da superfície:

$$n = \text{normalize}\left(x, \frac{\text{radius}}{\text{height}}, z\right)$$

onde **radius** é o raio da base e **height** é a altura do cone. As coordenadas de textura da lateral mapeiam o ângulo α horizontalmente e a altura y verticalmente:

$$u = \frac{\alpha}{2\pi}$$

$$v = \frac{y}{h}$$

2.2.2.5. Cilindro

As bases do cilindro recebem normais constantes apontando para cima ou para baixo consoante a base:

$$n_{\text{topo}} = (0, 1, 0)$$

$$n_{\text{base}} = (0, -1, 0)$$

As coordenadas de textura das bases são mapeadas circularmente, tal como no cone:

$$u = 0.5 + 0.5 \sin(\alpha)$$

$$v = 0.5 + 0.5 \cos(\alpha)$$

Para o ponto situado no centro das bases, as coordenadas de textura são:

$$u_{\text{centro}} = 0.5$$

$$v_{\text{centro}} = 0.5$$

Na superfície lateral, as normais são puramente radiais, sem componente vertical:

$$n = (\sin(\alpha), 0, \cos(\alpha))$$

As coordenadas de textura da lateral mapeiam o ângulo horizontalmente e a altura verticalmente entre 0 e 1:

$$u = \frac{\alpha}{2\pi}$$

$$v \in \{0, 1\}$$

2.2.2.6. Torus

Para o torus de `majorRadius` R e `minorRadius` r , a normal em cada vértice aponta radialmente para fora do centro do tubo. Dado o ponto parametrizado por θ (ângulo no círculo maior) e φ (ângulo no tubo), a normal é:

$$n = (\cos(\varphi) \cdot \cos(\theta), \sin(\varphi), \cos(\varphi) \cdot \sin(\theta))$$

As coordenadas de textura são obtidas normalizando diretamente os ângulos paramétricos:

$$u = \frac{\theta}{2\pi}$$

$$v = \frac{\varphi}{2\pi}$$

2.2.2.7. Fita de Möbius

Por ser uma superfície não orientável, as normais da fita de Möbius são calculadas analiticamente através das derivadas parciais da parametrização. Para cada ponto da superfície em (u, v) , calculam-se os vetores tangentes:

$$\begin{aligned}\frac{\partial x}{\partial u} &= -(r + v \cos(\frac{u}{2})) \sin(u) + \frac{v \sin(\frac{u}{2})}{2} \cos(u) \\ \frac{\partial y}{\partial u} &= (r + v \cos(\frac{u}{2})) \cos(u) - \frac{v \sin(\frac{u}{2})}{2} \sin(u) \\ \frac{\partial z}{\partial u} &= \frac{v}{2} \cos(\frac{u}{2}) \\ \frac{\partial x}{\partial v} &= \cos(\frac{u}{2}) \cos(u) \\ \frac{\partial y}{\partial v} &= \cos(\frac{u}{2}) \sin(u) \\ \frac{\partial z}{\partial v} &= \sin(\frac{u}{2})\end{aligned}$$

A normal é obtida pelo produto vetorial destes dois vetores tangentes e normalizada:

$$n = \text{normalize}\left(\frac{\partial}{\partial u} \times \frac{\partial}{\partial v}\right)$$

As coordenadas de textura são calculadas normalizando u e v para $[0, 1]$:

$$u_{\text{tex}} = \frac{u}{2\pi}$$

$$v_{\text{tex}} = \frac{v + \frac{\text{width}}{2}}{\frac{\text{width}}{2}}$$

2.2.2.8. Garrafa de Klein

Tal como na fita de Möbius, as normais da garrafa de Klein são calculadas analiticamente a partir das derivadas parciais da parametrização em relação a u e v , cujas expressões diferem entre os dois ramos da superfície. A normal é então obtida pelo produto vetorial:

$$n = \text{normalize}\left(\frac{\partial P}{\partial u} \times \frac{\partial P}{\partial v}\right)$$

Uma vez que a garrafa de Klein é uma superfície não orientável, a orientação da normal é verificada calculando o produto escalar com o vetor posição do vértice. Se o resultado for negativo, a normal é invertida para garantir que aponta aproximadamente para fora:

$$\text{se } n \cdot P < 0 \text{ então } n \leftarrow -n$$

As coordenadas de textura são calculadas normalizando diretamente os ângulos paramétricos:

$$u_{\text{tex}} = \frac{u}{2\pi}$$

$$v_{\text{tex}} = \frac{v}{2\pi}$$

2.2.2.9. Superfícies Bézier

As normais da superfície de Bézier são calculadas analiticamente através das derivadas parciais da equação bicúbica em relação a u e v . A derivada em ordem a u é obtida usando os polinómios de Bernstein de grau 2, aplicados às diferenças entre pontos de controlo consecutivos na direção de i :

$$\frac{\partial S}{\partial u} = \sum_{i=0}^2 \sum_{j=0}^3 3 \cdot B_i^2(u) \cdot B_j^3(v) \cdot (P_{i+1,j} - P_{i,j})$$

Já a derivada em ordem a v usa os polinómios de grau 2 na direção de j :

$$\frac{\partial S}{\partial v} = \sum_{i=0}^3 \sum_{j=0}^2 3 \cdot B_i^3(u) \cdot B_j^2(v) \cdot (P_{i,j+1} - P_{i,j})$$

A normal em cada vértice é então o produto vetorial das duas derivadas, normalizado:

$$n = \text{normalize}\left(\frac{\partial S}{\partial v} \times \frac{\partial S}{\partial u}\right)$$

Nos casos em que o vetor resultante é nulo, como acontece em pontos singulares onde as derivadas são paralelas, a normal é substituída pelo valor por defeito $(0, 1, 0)$. As coordenadas de textura mapeiam diretamente os parâmetros u e v , que já se encontram no intervalo $[0, 1]$:

$$u_{\text{tex}} = u \quad v_{\text{tex}} = v$$

3. Engine

No relatório da fase anterior, foi dito que ao clicar nas teclas **R** e **H** a curva de catmull-rom seriam renderizadas ou deixariam de ser renderizadas, respetivamente. Nesta fase, ao clicar na tecla **R** tanto os eixos do referencial como as curvas vão ser renderizadas se não estiverem renderizadas e vão deixar de ser se estiverem a ser renderizadas. Além disso, ao clicar na tecla **F** os limites do frustum podem ser renderizados ou deixar de ser renderizados, dependendo do estado ao clicar na tecla.

3.1. Extensão no formato XML

De forma a expandir a câmara orbital, foi adicionada uma expansão ao formato XML padrão usado no projeto. Esta extensão trata-se de permitir destacar certos grupos no ficheiro XML como "targets". Esses grupos "target" seriam passíveis de serem orbitados pela câmara orbital, em vez da mesma ficar sempre posicionada a orbitar a origem do referencial.

Foi decidido que os "targets" da câmara orbital seriam grupos em vez de modelos individuais, uma vez que modelos pertencentes ao mesmo grupo são construídos num referencial que sofre as mesmas transformações geométricas, logo não iria haver uma grande diferença entre orbitar modelos individuais de um mesmo grupo ou orbitar diretamente o grupo como um todo.

No excerto abaixo é possível ver parte de um ficheiro XML onde é usada a extensão criada. O atributo `target` do `group` é um atributo opcional do tipo `string` que permite identificar se esse grupo é um "target".

```
<!-- SOL -->
<group target="Sun">
  <transform>
    <scale x="5.5" y="5.5" z="5.5" />
    <rotate time="90" x="0" y="1" z="0" />
  </transform>
  <models>
    <model file="sphere_1_10_10.3d" />
  </models>

<!-- TERRA -->
<group target="Terra">
  <transform>
    <scale x="0.2" y="0.2" z="0.2" />
    <rotate time="4" x="0" y="1" z="0" />
  </transform>
  <models>
    <model file="sphere_1_10_10.3d" />
  </models>

<!-- Lua -->
<group>
  <transform>
    <scale x="0.2" y="0.2" z="0.2" />
    <rotate time="7" x="0" y="1" z="0" />
  </transform>
  <models>
```

```

        <model file="sphere_1_10_10.3d" />
      </models>
    </group>
  </group>
</group>

```

Como é possível ver no excerto, existem dois grupos que são "targets", sendo um deles identificado como "Sun" e o outro como "Terra". Ao clicar na tecla **1** e trocar para a câmara orbital, a câmara vai diretamente orbitar o primeiro grupo definido como "target", sendo, neste caso, o grupo identificado como "Sun". Para alternar entre "targets" é possível clicar na tecla **t** ou **T**, que vai mudando de "target" pela ordem de aparecimento dos mesmos no ficheiro XML, ou também clicar diretamente no objeto na tela, que vai mudar o "target" para o grupo ao qual o objeto clicado pertence, contando que esse grupo seja um "target". Neste caso, ao entrar na câmara orbital e clicar na tecla **t** ou **T**, a câmara orbital iria passar a orbitar o grupo identificado como "Terra". Como o grupo filho do grupo "Terra" não possui o atributo target, o mesmo não é passível de ser orbitado.

3.2. Atualização da Câmara Orbital

Tal como foi dito mais acima, a câmara orbital foi atualizada de forma à mesma ser capaz de orbitar múltiplos alvos. Para suportar esta nova funcionalidade, foi adicionada à câmara orbital uma referência ao grupo atualmente selecionado como alvo, assim como um índice, que ajuda a obter o próximo grupo a escolher, e o número total de grupos definidos como "target". A conversão de coordenadas esféricas para coordenadas cartesianas foi atualizada para ter em conta a posição global do grupo ($x_{\text{target}}, y_{\text{target}}, z_{\text{target}}$) após o mesmo sofrer as respetivas transformações geométricas e herdar também as transformações do grupo pai:

$$x = x_{\text{target}} \cdot r \cdot \cos(\beta) \cdot \sin(\alpha)$$

$$y = y_{\text{target}} \cdot r \cdot \sin(\beta)$$

$$z = z_{\text{target}} \cdot r \cdot \cos(\beta) \cdot \cos(\alpha).$$

O foco da câmara também é atualizado de forma à mesma estar sempre focada no grupo atualmente selecionado:

$$L_x = x_{\text{target}}$$

$$L_y = y_{\text{target}}$$

$$L_z = z_{\text{target}}$$

Assim como foi mencionado anteriormente, esta câmara poder acedida ao clicar na tecla **1** e a navegação entre alvos pode ser feita com as tecla **t** e **T**. Além disso, também implementamos uma segunda forma de navegação entre alvos, sendo ela feita ao clicar diretamente nas primitivas desenhadas na tela através de **picking**. Por exemplo, se um grupo possuir um modelo de uma esfera e esse grupo estiver definido como "target", ao clicar nessa mesma esfera com o rato a câmara vai mudar de alvo para o grupo da esfera. Esta funcionalidade funciona tanto na câmara orbital como na câmara normal disponível ao iniciar o engine.

3.3. View Frustum Culling

O frustum de visão constitui o volume da pirâmide truncada que representa o espaço visível pela câmara virtual colocada no espaço tridimensional. Ele é definido por 6 planos, os planos frontais **near** e **far** e os restantes planos laterais. A técnica de view frustum culling é uma otimização que consiste em não renderizar qualquer construção geométrica que não intersete este volume, uma vez que não será vista pelo observador.

O frustum pode ser representado por 6 planos do tipo $ax + by + cz + d$, onde o vetor normal (a, b, c) dita a orientação do plano, e o escalar d define a posição relativa à origem. Todas as normais dos 6 planos apontam para o interior do frustum. Para calcular a equação de cada plano são usados 3 pontos pertencentes ao plano, primeiro esses pontos são usados para determinar 2 vetores que constituam o plano, depois esses vetores são usados para calcular o produto vetorial e obter a normal do plano e, por fim, usa-se um dos pontos para determinar d garantindo que $ax + by + cz + d = 0$ para qualquer ponto do plano. Para determinar os 3 pontos a usar para o cálculo da equação de cada plano, usa-se a posição e o valor `lookAt` da câmara para determinar o vetor direção:

$$d = L - P$$

Depois normaliza-se d e o vetor up da câmara, obtém-se o vetor r através do produto vetorial de d e up normalizados.

$$r = d \times up$$

Após isso, normaliza-se r e faz-se o produto vetorial de r e d para obter u .

$$u = r \times d$$

Sabendo esses vetores, a posição da câmara, as distâncias aos planos **near** e **far**, assim como o **fov** e o **ratio**, obtidos direta ou indiretamente através do ficheiro XML inicial, somos capazes de calcular os pontos necessários para determinar as equações dos planos.

Durante a fase de renderização, cada modelo é testado para determinar se esse mesmo modelo deve ou não ser renderizado. O view frustum culling implementado neste projeto utiliza esferas para determinar se os modelos devem ser renderizados. Assim como foi referido na secção do **generator**, o ficheiro dos modelos contém informação sobre o raio e centro da esfera no espaço local. No entanto, como as equações dos planos do frustum são calculadas em espaço global, as esfera que englobam os modelos também devem ser testadas em espaço global. Para isso, é necessário transformar o raio e centro da esfera de cada modelo em espaço local para espaço global.

Como o centro e o raio globais devem ser calculados em espaço global, estes cálculos devem ser feitos antes de ser colocada a câmara, ou seja, não pode ser feito durante o processo de renderização. Para isso, antes da câmara ser colocada é realizado um processo semelhante ao que acontece durante a renderização, onde a hierarquia de grupos é iterada e, para cada grupo, é calculado o centro e o raio globais de cada modelo.

Este processo começa por usar as funções `glPushMatrix` e `glPopMatrix` para isolar os próximos cálculos, depois chama-se a função `glLoadIdentity` que substitui a matriz `MODELVIEW(MV)` pela matriz identidade, itera-se pela hierarquia de grupos aplicando as

transformações geométricas desse grupo, e, em cada grupo, calcula-se e armazena-se os raios e centros globais. Como apenas são aplicadas transformações, ao chamar a função `glPopMatrix`, tudo volta a como estava antes de se chamar `glPushMatrix` sem quaisquer alterações visuais para o observador.

Em cada grupo, durante o processo explicado acima e após a aplicação das transformações desse mesmo grupo, para calcular o raio e centro globais, começa-se por obter a matriz `MODELVIEW(MV)` com `glGetFloatv`, onde, devido à função `glLoadIdentity` chamada antes de se começar a aplicação das transformações, a `MODELVIEW(MV)` apenas contém informações da matriz `MODEL(M)`. Para cada modelo, ao multiplicar o centro local pela matriz `MODELVIEW(MV)` obtém-se o centro global, e ao extrair as escalas dessa mesma matriz, multiplica-se o raio local pela escala máxima entre as escalas (`sx, sy, sz`), de forma a garantir que a esfera resultante do centro e raio globais engloba o modelo correspondente.

Após obter o centro e raio globais de todos os modelos da hierarquia de grupos, começa-se realmente o processo de renderização, onde para cada modelo é feita uma verificação: a esfera que engloba o modelo interseção o frustum se, para todos os seis planos, a distância do centro da esfera (`cx, cy, cz`) ao plano for superior ou igual a $-r(\text{raio})$, isto é:

$$a \cdot c_x + b \cdot c_y + c \cdot c_z + d \geq -r$$

Se esta condição falhar para qualquer um dos seis planos, o modelo não é renderizado.

O processo de obtenção das equações dos 6 planos que compõem o frustum e a obtenção do raio e centro globais é realizado cada vez que a função `renderScene` é chamada para renderizar uma nova frame.

3.4. Picking

Como já foi mencionado, o nosso projeto implementa `picking` de forma a permitir navegação entre alvos na câmara orbital. Em primeiro lugar, a posição do clique (`mouseX, mouseY`) é ajustada de tal forma que o `mouseY` é invertido, uma vez que a origem do referencial da janela do sistema operativo (a área de visualização) fica no canto superior esquerdo, enquanto que no `OpenGL` essa origem está no canto inferior esquerdo. Após a posição do clique ser ajustada, a mesma é usada para ler a profundidade do pixel correspondente através da função `glReadPixels` com `GL_DEPTH_COMPONENT`, sendo então a profundidade usada como `z` nas coordenadas da janela. Se o valor de profundidade for 1.0, significa que o clique foi muito longe na cena, sendo o mesmo imediatamente invalidado.

O segundo passo é gerar as coordenadas normalizadas do dispositivo (NDC) através das coordenadas da janela (`x, y, z`) obtidas anteriormente. Para isso é necessário o `viewport` que é obtido através de `glGetIntegerv(GL_VIEWPORT, viewport)`, que nos permite obter um vetor de 4 inteiros.

Sabendo as coordenadas da janela e o vetor `viewport` podemos calcular as coordenadas NDC:

$$x_{\text{NDC}} = 2 \cdot \frac{x_{\text{window}}}{\text{viewport}_2} - 1$$

$$y_{\text{NDC}} = 2 \cdot \frac{y_{\text{window}}}{\text{viewport}_3} - 1$$

$$z_{\text{NDC}} = 2 \cdot z_{\text{window}} - 1$$

Após obter as coordenadas **NDC**, as matrizes **MODELVIEW(MV)** e **PROJECTION(P)** são lidas com `glGetFloatv`. Depois disso, as matrizes são então multiplicadas para obter a matriz composta $P \cdot MV$, sendo a mesma invertida. As coordenadas **NDC** são multiplicadas pela matriz calculada anteriormente e depois divididas pela componente **w**, obtendo então as coordenadas do ponto no espaço global.

Tendo as coordenadas do espaço global, podemos usar essas coordenadas para ver se as mesmas estão dentro de alguma das esferas que envolvem os modelos pertencentes a um grupo "**target**". O processo começa por iterar pelos grupos definidos como "**target**", e para cada modelo pertencente a esse grupo calcular a distancia entre o ponto calculado e o centro global da esfera desse modelo. Se essa distancia for menor ou igual ao raio global da esfera do modelo e menor que a menor distância encontrada até ao momento, o grupo do qual esse modelo faz parte é marcado como selecionado. Isto garante que, no caso de haver sobreposição entre modelos de diferentes grupos, apenas o grupo do modelo mais próximo é selecionado. Após todos os grupos "**target**" terem sido iterados, o grupo que estiver marcado como selecionado é usado como novo "**target**" na câmara orbital. Se nenhum grupo for selecionado, a câmara não muda.

A pipeline de renderização segue um percurso ordenado: vértices no espaço local do objeto são transformados para o espaço de câmara com a matriz **MODELVIEW**, depois para o espaço de **clip** com a matriz **PROJECTION**, submetidos à divisão de perspectiva (divisão pela componente **w**) para serem transformados em coordenadas **NDC** e finalmente mapeadas para pixels na janela de visualização através da transformação do **viewport**. O **picking** implementado neste projeto inverte este percurso de forma a conseguir obter coordenadas do espaço global através da janela de visualização e da profundidade do pixel selecionado.

3.5. Iluminação

A iluminação no **OpenGL** é configurada segundo uma ordem fixa, na qual a iluminação ambiente global é ativada e, em seguida, as fontes de luz disponíveis são instanciadas e parametrizadas, sendo definidas as parcelas difusa, especular e ambiente, sendo estes parâmetros responsáveis por definir como a iluminação irá interagir com os materiais dos objetos. Este processo inicial apenas é executado uma vez no arranque do **engine** e prepare o estado da iluminação para o que acontece durante a renderização.

Neste projeto, são suportadas 3 tipo de modelos de iluminação: luzes posicionais, direcionais e focos luminosos. Estes modelos são usados para definir o comportamento da iluminação durante o processo de renderização. Cada uma das fontes de iluminação instanciadas no arranque do **engine** irá usar um destes modelos durante a renderização.

Durante a renderização, as luzes posicionais representam um ponto localizado no espaço tridimensional, a sua intensidade depende da distância entre a sua localização e o ponto da superfície que está a iluminar, o que a torna adequada para simular lâmpadas ou outro tipo de emissor com posição definida. As luzes direcionais representam uma fonte sem posição de origem efetiva, apenas uma direção. Os raios luminosos produzidos por

este modelo são usados para aproximar fenômenos com a luz do sol. Por fim, o foco luminoso combina uma posição de onde a luz se origina com uma direção de iluminação. Esta luz é confinada a um cone de iluminação delimitada por um ângulo de corte. Estas características fazem com que apenas objetos dentro do cone luminoso recebam iluminação da fonte.

Outro fator importante na iluminação são as normais das superfícies que estão a ser iluminadas, uma vez que elas definem a forma como a luz interage com ela. Por exemplo, a luz difusa depende do ângulo entre a normal e a direção da luz e a luz especular depende da normal, da direção da luz e da direção de observação.

No projeto, assim como acontece com os vértices, as normais dos modelos de um grupo são guardadas ordenadamente num **VBO** armazenado nesse grupo, estando o conteúdo armazenado na memória da **GPU**. Durante o processo de renderização as normais são chamadas juntamente com os vértices e as coordenadas de textura no momento de renderização de um modelo. Além disso, como os modelos podem sofrer transformações geométricas, é importante que as normais sejam recalculadas de forma à iluminação se manter consistente. Este recálculo é feito com a chamada da função `glEnable(GL_RESCALE_NORMAL)` na inicialização da iluminação no arranque do **engine**.

3.6. Texturas

As texturas tornam possível introduzir variação visual da superfície dos objetos no sistema de renderização, sendo tratadas como mapas de cor bidimensionais aplicados às geometrias do modelo. Ao arrancar o **engine**, durante a iteração pelo ficheiro XML e ao fazer parsing dos modelos de cada grupo, a textura de cada modelo, se existir, é inicialmente carregada a partir de uma biblioteca de tratamento de imagens externa (**DevIL**), convertida para um formato compatível com a unidade gráfica e transferida para a memória da **GPU**, onde são geradas as **mipmaps** e definidos os parâmetros de amostragem e repetição. Estes parâmetros controlam o nível de detalhe e o comportamento das coordenadas que excedem o intervalo unitário, garantindo resposta adequada a diferentes escalas de projeção e distâncias da câmara.

As coordenadas de textura (u, v), responsáveis por mapear a textura para a superfície dos modelos, são tratadas de forma análoga às posições e normais. Para otimizar o desempenho, o **engine** agrega as coordenadas de textura relativas aos modelos que pertencem a um mesmo grupo de forma ordenada e armazena-as num **VBO** cujo conteúdo está guardado na **GPU**. Quando um grupo está a ser renderizado, o buffer de coordenadas de textura correspondente é vinculado e apontado como fonte de dados para os atributos de coordenadas de textura.

Durante a etapa de renderização, as primitivas são então desenhadas utilizando os mesmos índices/ordens dos vértices e normais, o que assegura que cada vértice irá estar vinculado à coordenada de textura correta.

4. Geração do Sistema Solar Dinâmico com Iluminação e Texturas

Nesta última fase, atualizamos a demonstração da fase anterior de forma a introduzir iluminação e texturas, permitindo a criação de um sistema solar ainda mais realista. Todos os objetos, com a exceção as primitivas usadas na construção do cinturão de asteroides, possuem uma textura. Resolvemos não usar texturas nos asteroides, uma vez que os mesmos são demasiado pequenos para a textura poder ser visualizada. Em termos de iluminação resolvemos usar apenas uma fonte de luz posicional com localização na origem do referencial, de forma a simular a existência do sol, que também é construído na origem.

Abaixo é possível ver um exemplo da utilização das texturas e da iluminação na representação de um modelo em xml:

```
<lights>
  <light type="point" posx="0" posy="0" posz="0" />
</lights>

<!-- SOL -->
<group>
  <transform>
    <scale x="5.5" y="5.5" z="5.5" />
    <rotate time="90" x="0" y="1" z="0" />
  </transform>
  <models>
    <model file="sphere_1_50_50.3d">
      <texture file="8k_sun.jpg" />
      <color>
        <diffuse R="255" G="255" B="255" />
        <ambient R="80" G="60" B="0" />
        <specular R="0" G="0" B="0" />
        <emissive R="255" G="180" B="40" />
        <shininess value="0" />
      </color>
    </model>
  </models>

  <!-- TERRA -->
  <group>
    <transform>
      <translate time="16" align="true">
        <point x="-2.37" y="0" z="1.99" />
        <point x="-3.08" y="0" z="-0.27" />
        <point x="-1.99" y="0" z="-2.37" />
        <point x="0.27" y="0" z="-3.08" />
        <point x="2.37" y="0" z="-1.99" />
        <point x="3.08" y="0" z="0.27" />
        <point x="1.99" y="0" z="2.37" />
        <point x="-0.27" y="0" z="3.08" />
      </translate>
      <scale x="0.2" y="0.2" z="0.2" />
      <rotate time="4" x="0" y="1" z="0" />
    </transform>
  </group>
</group>
```

```
</transform>
<models>
  <model file="sphere_1_50_50.3d">
    <texture file="2k_earth_daymap.jpg" />
  </model>
</models>
<!-- Lua -->
<group>
  <transform>
    <translate time="3" align="true">
      <point x="2.300" y="0" z="0.000" />
      <point x="1.626" y="0" z="1.626" />
      <point x="0.000" y="0" z="2.300" />
      <point x="-1.626" y="0" z="1.626" />
      <point x="-2.300" y="0" z="0.000" />
      <point x="-1.626" y="0" z="-1.626" />
      <point x="0.000" y="0" z="-2.300" />
      <point x="1.626" y="0" z="-1.626" />
    </translate>
    <scale x="0.2" y="0.2" z="0.2" />
    <rotate time="7" x="0" y="1" z="0" />
  </transform>
  <models>
    <model file="sphere_1_50_50.3d">
      <texture file="2k_moon.jpg" />
    </model>
  </models>
</group>
</group>
```

Nas figuras a seguir é possível ver a demonstração construída nesta fase:



Figura 2: Parte da demonstração sem as curvas de catmull-rom não visíveis



Figura 3: Parte da demonstração sem as curvas de catmull-rom não visíveis

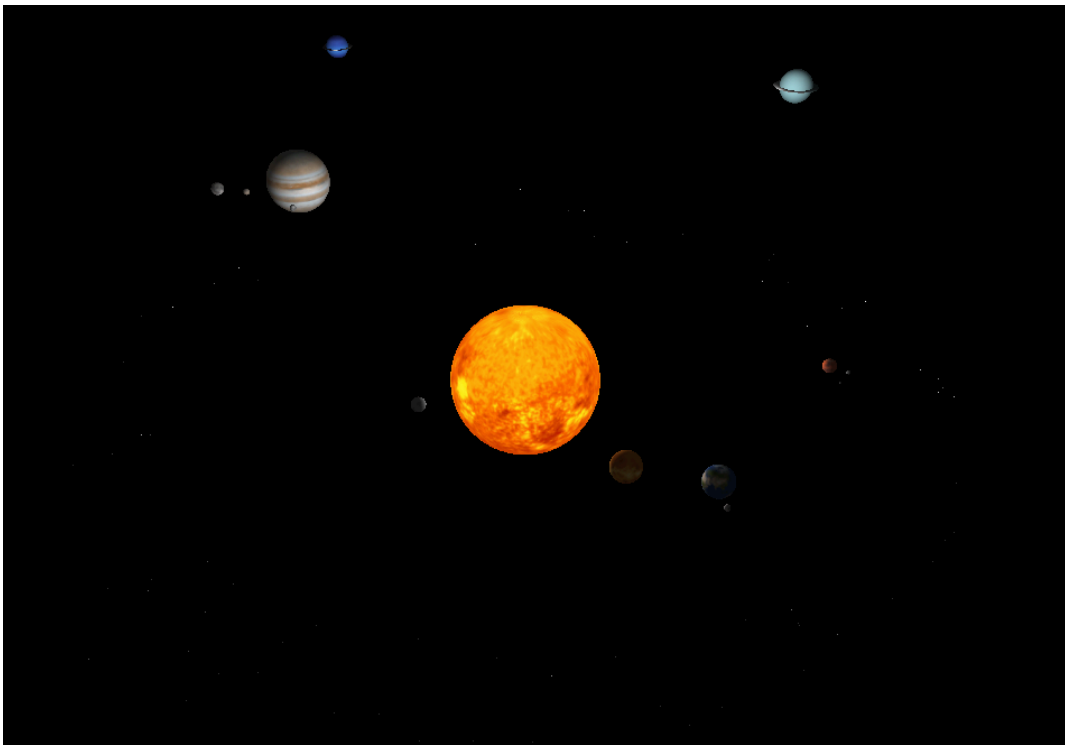


Figura 4: Parte da demonstração sem as curvas de catmull-rom não visíveis

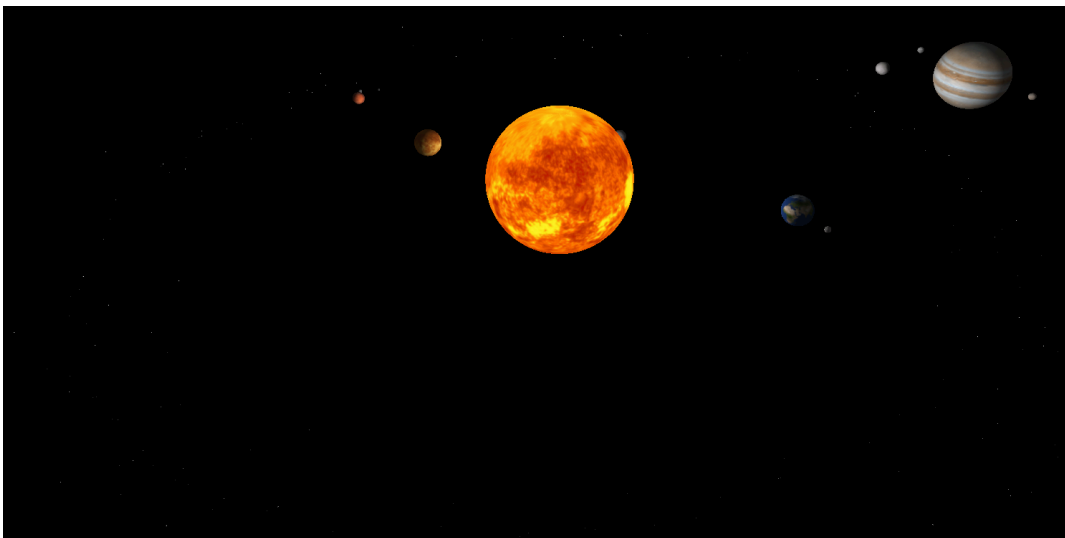


Figura 5: Parte da demonstração sem as curvas de catmull-rom não visíveis



Figura 6: Parte da demonstração sem as curvas de catmull-rom não visíveis

5. Resultados Obtidos

Assim como nas fases anteriores, a equipa docente disponibilizou outputs de referência de modo a conseguirmos comparar os nossos resultados. Nas seguintes figuras é possível ver a comparação entre os outputs obtidos e os disponibilizados:

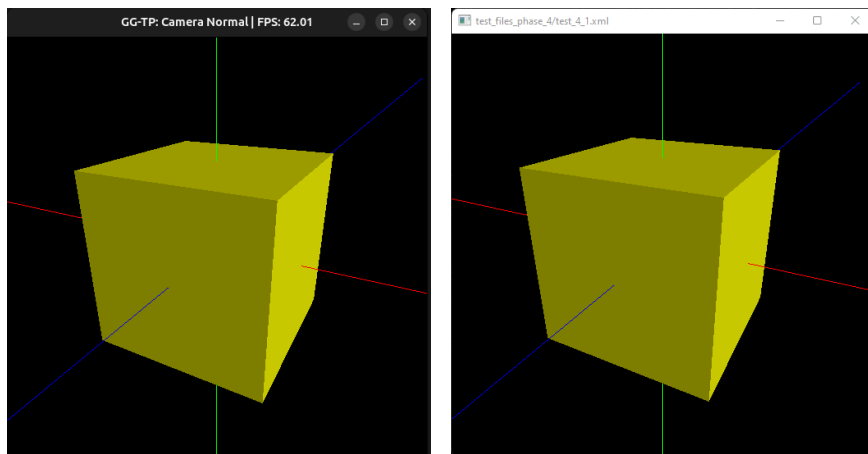


Figura 7: Comparação dos outputs obtidos no teste 1

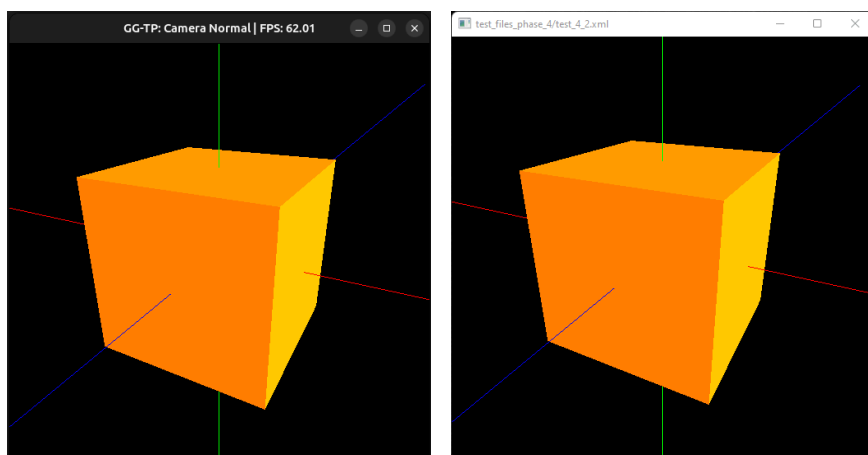


Figura 8: Comparação dos outputs obtidos no teste 2

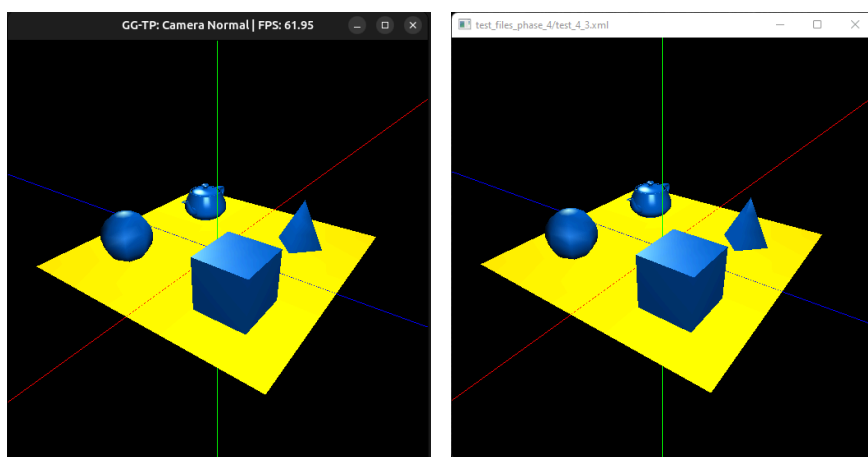


Figura 9: Comparação dos outputs obtidos no teste 3

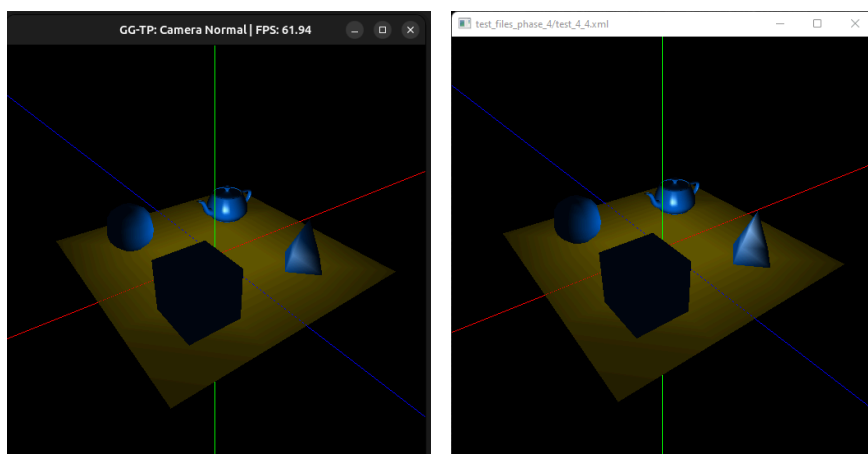


Figura 10: Comparação dos outputs obtidos no teste 4

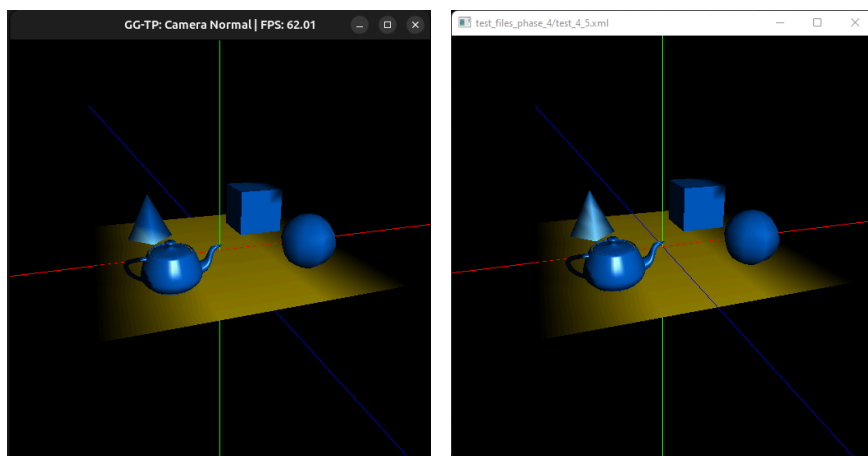


Figura 11: Comparação dos outputs obtidos no teste 5

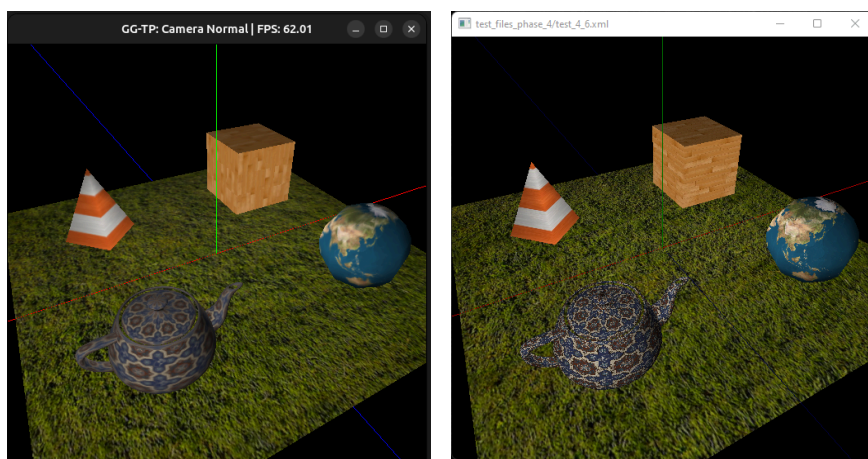


Figura 12: Comparação dos outputs obtidos no teste 6

Nas figuras mais abaixo, é possível ver mais alguns exemplos de cenas produzidas para testar a aplicação de texturas e iluminação no engine:

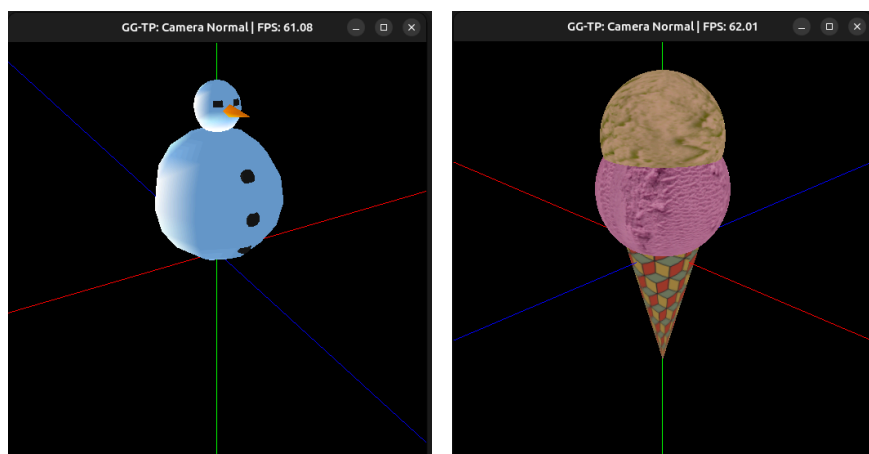


Figura 13: Outros exemplos de cenas produzidas

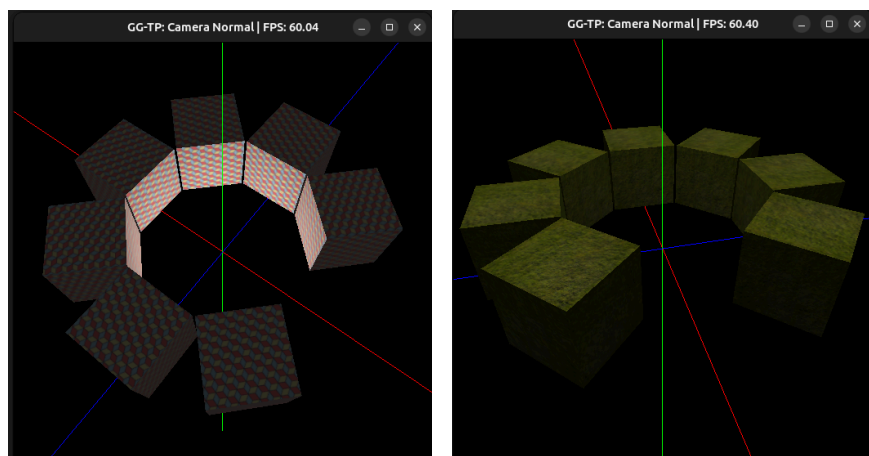


Figura 14: Outros exemplos de cenas produzidas

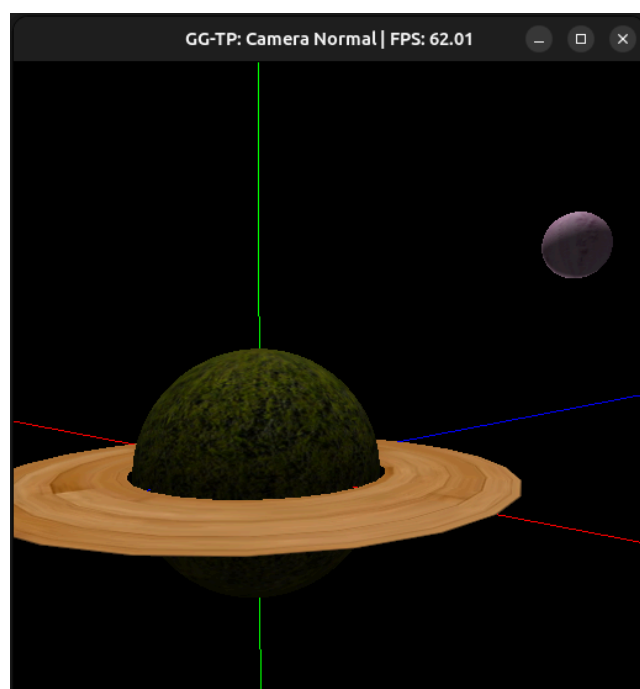


Figura 15: Outro exemplo de cenas produzidas

6. Conclusão

A realização da quarta fase deste projeto permitiu aprofundar a nossa compreensão de dois dos conceitos mais importantes na área da Computação Gráfica: a iluminação e as texturas. A introdução destes conceitos tornou possível a representação de objetos com um maior nível de realismo visual, aproximando a aparência das superfícies e a interação da luz ao comportamento observado no mundo real.

Para suportar estas novas funcionalidades, foi necessário expandir o **generator** de forma a produzir, para todas as primitivas implementadas, as respectivas normais e coordenadas de textura. No **engine**, foram adicionados mecanismos de iluminação capazes de suportar luzes posicionais, direcionais e focos luminosos, bem como o carregamento e aplicação de texturas através da utilização de **VBOs** para vértices, normais e coordenadas de textura. Estas alterações permitiram melhorar significativamente a qualidade visual das cenas produzidas, mantendo simultaneamente um desempenho adequado durante a renderização.

Os conhecimentos adquiridos ao longo desta fase foram aplicados na atualização da demonstração do Sistema Solar desenvolvida na fase anterior, que passou a incluir iluminação e também texturas específicas para os diferentes corpos celestes, resultando numa representação mais detalhada e realista. Para além disso, foram implementadas funcionalidades adicionais, nomeadamente o view frustum culling baseado em esferas, a navegação da câmara orbital entre múltiplos alvos através de "**targets**" e **picking**, e uma versão mais configurável da garrafa de Klein.